Pearls AQI Predictor

A Cloud-Based Machine-Learning System for
3-Day Air-Quality Forecasting in Karachi

Internship Technical Report

Author: Muhammad Usman
Contact: muusman4@gmail.com
Live demo: aqi-predictor.streamlit.app
Repository: `aqi-predictor` (GitHub)

June 7, 2026

Abstract

This report documents an end-to-end, fully cloud-based machine-learning system that forecasts the Air Quality Index (AQI) for Karachi over a three-day horizon (24 h / 48 h / 72 h). The system ingests pollutant concentrations from OpenWeather and meteorology from OpenWeather and Open-Meteo (with AQICN used as an independent live cross-check), derives the AQI from $PM_{2.5}$ with the US-EPA formula, engineers roughly fifty features, stores everything in a MongoDB Atlas feature store, and retrains a suite of classical, gradient-boosting, ensemble and deep-learning models every day. A TOPSIS-based champion–challenger gate selects and registers the best model, whose precomputed forecasts are served through an interactive Streamlit dashboard. Both pipelines run autonomously on GitHub Actions (hourly feature ingestion, daily retraining) with no servers to manage. The current champion — a voting ensemble — attains a test-set RMSE of ≈28–38 AQI points across the three horizons, beating the persistence baseline by a wide margin at every horizon. The report covers all six required deliverables: the feature pipeline, historical backfill, training pipeline, automated CI/CD, the web dashboard, and the advanced-analytics layer (EDA, SHAP explainability, hazardous-AQI alerts, conformal prediction intervals and feature ablation).

Contents

1 Introduction

Air pollution is a chronic public-health problem in Karachi, where fine particulate matter ($PM_{2.5}$) routinely pushes the AQI into the *unhealthy* and *hazardous* ranges. Reliable short-term forecasts let residents and authorities anticipate bad-air days and take precautions.

The goal of this project was to build a **production-grade, cloud-based forecasting system** that predicts Karachi's AQI for the next three days and presents it through a public dashboard, while exercising the full MLOps lifecycle: data ingestion, feature engineering, a feature store, model training and registry, automated retraining, monitoring, and deployment.

Deliverables. The system was specified around six deliverables, each of which this report addresses in a dedicated section:

1. **Feature pipeline** — fetch raw weather/pollutant data and compute time-based and derived features (Section 3).
2. **Historical backfill** — generate a training dataset from past dates (Section 4).
3. **Training pipeline** — train and evaluate multiple models, register the best (Section 5).
4. **Automated CI/CD** — hourly feature runs and daily retraining (Section 6).
5. **Web dashboard** — interactive real-time forecast UI (Section 7).
6. **Advanced analytics** — EDA, explainability, alerts and multi-model forecasting (Section 8).

A mapping from each deliverable to its concrete implementation is given in Appendix A.

2 System Architecture

The system is *fully cloud-based*: every component runs on a managed cloud service, with no always-on server to provision or maintain. Two scheduled GitHub Actions workflows do all the compute, MongoDB Atlas is the single source of truth for data and models, and the Streamlit Community Cloud dashboard reads precomputed results on demand. Figure 1 shows the data flow.

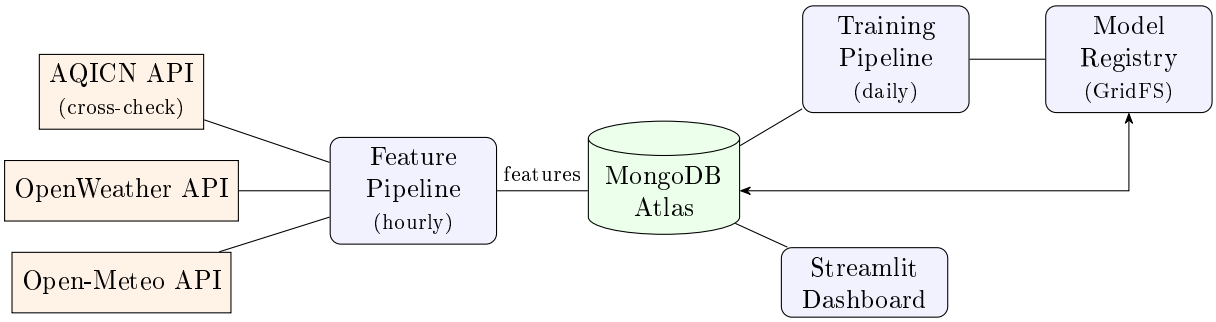


Figure 1: Cloud-based data flow. The feature pipeline (GitHub Actions, hourly) writes engineered features to MongoDB; the training pipeline (daily) reads them, trains the model suite, and registers the champion's artifacts in GridFS; the dashboard reads precomputed forecasts and leaderboard metadata from MongoDB.

Table 1: Technology stack and the role of each component.

Layer	Technology	Role
Data sources	OpenWeather, Open-Meteo, AQICN	Pollutant $\mu g/m^3$, weather, AQI cross-check
Feature store	MongoDB Atlas (`aqi_features`)	Engineered feature rows
Model registry	MongoDB + GridFS	Metrics, history, champion binaries
Orchestration	GitHub Actions	Hourly + daily scheduled runs
Modelling	scikit-learn, XGBoost, LightGBM, CatBoost, TensorFlow	Model suite
Explainability	SHAP, MAPIE (conformal)	Importances, prediction intervals
Dashboard	Streamlit + Plotly	Public interactive UI
Experiment log	MLflow	Per-run tracking artifacts

Why this cloud architecture? Cost (everything runs on managed cloud free tiers), reproducibility (each run is a clean, logged cloud container), and zero operational overhead — no servers to provision, patch, or keep online. The dashboard never performs live training or heavy inference — it only reads forecasts the feature pipeline precomputed hourly — so it stays responsive even on Streamlit Community Cloud.

3 Feature Pipeline Development

The feature pipeline runs every hour. It (1) fetches the latest observation, (2) engineers features, and (3) upserts a single row keyed by `(city, timestamp)` into the `aqi_features` collection.

3.1 Raw data acquisition

`src/feature_pipeline/fetch_data.py` draws on three free providers, each in a specific role:

- **OpenWeather** `/air_pollution` (current endpoint live, `/history` for backfill) — true pollutant concentrations in $\mu g/m^3$ ($PM_{2.5}$, PM_{10}, O_3, NO_2, SO_2, CO). *This is the source from which the stored AQI is computed* (Section 3.2).
- **OpenWeather** `/weather` — live meteorology (temperature, humidity, pressure, wind speed/direction, cloud cover, precipitation).
- **Open-Meteo Archive API** (`archive-api.open-meteo.com`) — hourly *historical* meteorology for the backfill (Section 4); free and key-less, it supplies the weather columns for every past timestamp.
- **Open-Meteo Forecast + CAMS air-quality API** — forward-looking weather and $PM_{2.5}$ for the `forecast_leads` group (currently disabled, Section 3.3).
- **AQICN** `/feed` — the station AQI for Karachi, retained purely as an *independent live cross-check* (stored as `aqi_aqicn_raw`); it is *not* used as a feature or target. The two scales diverge for Karachi (AQICN reports ≈161 where OpenWeather $PM_{2.5} = 16.6\,\mu g/m^3$ gives a US-EPA AQI of ≈61, matching independent monitors), so mixing them would be inconsistent — hence a single, reproducible $PM_{2.5}$→AQI definition is used everywhere.

Why split the sources this way? Each is chosen for the one thing it does best and for free: OpenWeather is the only one of the three that returns *true* pollutant concentrations in $\mu g/m^3$ (which the EPA formula needs), Open-Meteo's archive gives key-less hourly history to seed the backfill, and AQICN is kept only as an independent sanity-check because its composite AQI is

on a different scale and would corrupt the targets if mixed in. Computing the AQI ourselves from one source keeps training and serving identical.

3.2 AQI definition (US-EPA)

The AQI is never taken from an API; it is computed deterministically from $PM_{2.5}$ so that training, serving and backfill share one definition (`pm25_to_aqi` in `feature_engineering.py`). The US-EPA AQI is a piecewise linear interpolation between concentration breakpoints:

$$\mathrm{AQI} \;=\; \frac{I_{\mathrm{hi}} - I_{\mathrm{lo}}}{C_{\mathrm{hi}} - C_{\mathrm{lo}}}\,\big(C - C_{\mathrm{lo}}\big) + I_{\mathrm{lo}}, \tag{1}$$

where C is the $PM_{2.5}$ concentration and $(C_{\mathrm{lo}}, C_{\mathrm{hi}}) \to (I_{\mathrm{lo}}, I_{\mathrm{hi}})$ is the breakpoint interval containing C (Table 2). Critically, the EPA index is defined on a **24 h-averaged** concentration, not an instantaneous reading, so the pipeline feeds Eq. (1) the *24 h trailing mean* of $PM_{2.5}$ (`pm25_24h`, a `rolling(24)` mean). This averaging removes most of the hour-to-hour spikiness that makes an instantaneous target hard to forecast. The three targets `aqi_24h/48h/72h` are exactly this quantity evaluated on the future 24 h-mean $PM_{2.5}$ at t+24/48/72 h.

Table 2: US-EPA $PM_{2.5}$ breakpoints used in Eq. (1) ($\mu g/m^3$).

C_{lo}	C_{hi}	I_{lo}	I_{hi}	**Category**
0.0	12.0	0	50	Good
12.1	35.4	51	100	Moderate
35.5	55.4	101	150	Unhealthy (sensitive)
55.5	150.4	151	200	Unhealthy
150.5	250.4	201	300	Very Unhealthy
250.5	500.4	301	500	Hazardous

3.3 Feature engineering

`src/feature_pipeline/feature_engineering.py` expands each observation into roughly fifty features, organised into nine groups (Table 3, the authoritative `FEATURE_GROUP_COLS` mapping in code). They span the **time-based features** (hour, day-of-week, month, season) and the **derived features** the brief requires — notably the AQI *change rate* — alongside multi-scale lags, rolling statistics, and physics-inspired cross features.

Stored vs. used for training. An important distinction: *every* group is always computed and written to `aqi_features` — the backfill forces all groups on so the stored schema is complete and never changes shape. The "In training" column in Table 3 is a separate decision: it says whether that group is actually *fed to the models* as an input. One group is deliberately held out of training: `forecast_leads` (it would leak the targets — see below). Beyond that, the ablation study (Section 8) can switch any remaining group off to measure its contribution, and that selection is persisted in the `feature_overrides` collection so it survives across GitHub Actions runs.

Table 3: Engineered feature groups (config flag `FEATURE_GROUPS`).

Group	Columns	In training
`time_basic`	`hour`, `month`, `season`, `is_weekend`, one-hot `dow_0`…`dow_6`	yes
`time_fourier`	`hour_sin`, `hour_cos`, `month_sin`, `month_cos`	yes
`lag_features`	`aqi_lag_{1,3,6,12,24,48,72,168}h`, `pm25_lag_{1,3,6,12,24}h`	yes
`rolling_stats`	`rolling_mean_{3,6,24}h`, `rolling_std_24h`, `rolling_min_24h`, `rolling_max_24h`, `aqi_pct_change_24h`	yes
`physics_derived`	`mixing_height_proxy`, `wind_transport_effect`, `temp_humidity_interaction`, `pm_ratio`, `wind_dir_sin`, `wind_dir_cos`	yes
`cross_feature`	`aqi_change_rate`, `pressure_anomaly`	yes
`raw_pollutants`	`pm25`, `pm10`, `o3`, `no2`, `so2`, `co`	yes
`meteorology`	`temperature`, `humidity`, `pressure`, `wind_speed`, `visibility`, `cloud_cover`, `precipitation_1h`	yes
`forecast_leads`	forecasted weather + $PM_{2.5}$ at t+24/48/72 h	**disabled**

Derived-feature definitions. The physics-inspired and cross features are simple, interpretable combinations rather than black-box transforms:

- `aqi_change_rate` $= \mathrm{AQI}_t - \mathrm{AQI}_{t-1}$ (the required change rate); `aqi_pct_change_24h` $= (\mathrm{AQI}_t - \mathrm{AQI}_{t-24})/\max(|\mathrm{AQI}_{t-24}|, 1)$.
- `mixing_height_proxy` $= 1000 \cdot T/\max(P, 1)$ — a cheap stand-in for boundary-layer mixing; `wind_transport_effect` $=$ wind_speed $\cdot$ pm25; `pm_ratio` $=$ pm10$/\max($pm25$, 0.01)$; `pressure_anomaly` $= P_t - \overline{P}_{24h}$.
- Cyclic encodings $\mathrm{x}_{\sin/\cos} = \sin/\cos(2\pi x/\text{period})$ for hour (24), month (12) and wind direction (360°), so the model sees their wrap-around continuity.

Lag and rolling features are computed from a trailing history window long enough to cover the 168 h (7-day) lag; missing history positions default to 0.
Why these features? They encode the structure the EDA actually shows in the data: AQI is strongly autocorrelated and repeats on daily and weekly cycles, so multi-scale **lags** (up to 168 h) and **rolling** statistics give the model that recent history directly; **Fourier** encodings let it use time-of-day and month as smooth, wrap-around signals (so 23:00 and 00:00 are adjacent); and the **physics** features are cheap proxies for how weather disperses or traps pollution (mixing, wind transport). All are simple, inspectable quantities — which is also what makes the SHAP explanations (Section 8) readable.
The `forecast_leads` group is deliberately disabled: during backfill those columns would be filled with *realised* future values, making them identical to the targets (target leakage). They will be re-enabled once the backfill is switched to an issued-forecast archive.

3.4 Normalisation

Normalisation is a three-stage, leakage-safe pipeline applied identically by the backfill seeder and the hourly pipeline (the shared `sanitize.py` module guarantees this):

1. **Input clamping.** Every raw field is clamped to a plausible physical range (`clean_raw_inputs`, e.g. $PM_{2.5} \in [0, 1000]$, pressure $\in [800, 1100]$ hPa, humidity $\in [0, 100]$%); non-finite or un-parseable values are dropped and later filled with 0 (`finalize_feature_row`), so a sensor glitch can never poison a row.

2. **Log transform.** Right-skewed pollutants ($PM_{2.5}$, PM_{10} and their lags) receive a $\log(x+1)$ transform to compress the heavy tail before scaling.
3. **Robust scaling + clip.** All numeric columns are scaled with a `RobustScaler` (median/IQR, resistant to AQI spikes); at training time the scaled matrix is additionally clipped to $[-8, 8]$ to bound extreme outliers (`X = np.clip(X, -8, 8)`).

Crucially, the scaler is *fit on the training split only* and persisted (`scaler_bundle.pkl`); the hourly pipeline loads this pre-fit scaler and only the daily training run re-fits it, so inference always applies the identical transform — no leakage from future statistics. `apply_scaler` tolerates feature-set changes (a scaler fit on an older column set still transforms correctly).

3.5 Targets

Each row carries three regression targets — `aqi_24h`, `aqi_48h`, `aqi_72h` — the realised AQI 24/48/72 h later. Because that future is unknown when a row is first written, the targets start null and are filled in once observed: the hourly pipeline back-writes the current AQI as the target for the rows from 24/48/72 h ago, and the daily `fill_targets.py` job (Section 4) backstops any that remain null. All models predict the three horizons simultaneously (multi-output, Section 5).

4 Historical Data Backfill

A useful model needs history, so the feature store is seeded once with the past year-plus of data before the hourly pipeline takes over.

- `src/backfill/backfill_features.py` pulls historical observations for the past `BACKFILL_DAYS` (default 365, set to 730 for a one-time 2-year re-seed) and upserts one engineered row per timestamp — the same schema the hourly pipeline produces. Pollutant history comes from OpenWeather `/air_pollution/history`, fetched in **180-day chunks** (with a 1 s pause between chunks) to avoid truncation/rate-limiting on long ranges; the matching hourly meteorology is left-joined from the **Open-Meteo Archive API**.
- `src/backfill/fill_targets.py` populates the `aqi_24h/48h/72h` targets for older rows once their "future" has actually been observed.
- `src/backfill/ablation_backfill.py` compares the missing-value imputation strategies (Section 4.1) so the training pipeline starts from a clean, gap-free dataset.

The result is the `aqi_features` collection used throughout the project: **15,776 hourly rows spanning 2024-05-30 to 2026-06-06** (≈2 years) at the time of writing. Because the backfill upserts by `(city, timestamp)`, it is idempotent and doubles as a schema-migration tool — re-running it after a schema change back-fills new columns without duplicating rows.

4.1 Missing-value imputation

Sensor feeds drop out, so the backfill must fill gaps before training. Five strategies are implemented (`backfill_features.py`); each operates column-by-column on the numeric fields:

- `forward_fill` — carry the last observation forward.
- `linear` — linear interpolation in both directions.
- `seasonal` — fill each gap from the recurring 24 h daily pattern (a seasonal decomposition with a 24 h period), so a missing hour is reconstructed from the same hour on surrounding days rather than a flat line.

- `knn` — `KNNImputer` with $k = 5$ across the numeric feature matrix.
- `hybrid` **(default)** — gap-length aware: ≤ 3 **h** filled linearly, **3–24 h** by the seasonal daily-pattern fill, > 24 **h** by KNN ($k = 5$); a column that is entirely missing (sensor fully offline) is set to 0.

Why gap-length aware? A one- or two-hour dropout is well covered by simple interpolation, but a multi-day outage has no nearby points to interpolate from — there the daily-pattern and neighbour-based (KNN) fills are far safer than stretching a straight line across the hole. The `hybrid` default applies each method only on the gap sizes it suits.

Choosing the strategy. `ablation_backfill.py` is not just a description — it empirically picks the winner. Over a *fixed* window (2025-11-01 to 2026-01-31, for a fair comparison) it backfills with each strategy, trains a fixed `RandomForestRegressor` (100 trees, depth 10), evaluates 24 h-target RMSE with a 3-fold `TimeSeriesSplit`, and selects the lowest-RMSE strategy. Scores are logged to MLflow and persisted to the `ablation_results` collection so the dashboard's Ablation tab (Section 7) shows the comparison live; this runs nightly as the first step of the training workflow.

5 Training Pipeline Implementation

The training pipeline (`src/training_pipeline/`) runs daily. It reads the feature store, trains a suite of models, evaluates them on a held-out temporal test set with out-of-fold cross-validation, ranks them, and registers the best.

5.1 Data & temporal splits

The pipeline reads the feature store via `fetch_training_data()` and trains on the **most recent ≈8,760 rows (≈1 year, `MAX_TRAIN_ROWS`)** of the ≈15,776-row history — a sliding one-year window that keeps the model on the current seasonal regime and bounds runtime. Because this is a time series, **all splits are chronological — never shuffled**:

- **Train / validation / test** = 70/10/20, split by time so the test set is strictly the most recent slice (`df.iloc[:n_train]`, etc.). The earliest 70 % trains; the final 20 % is never seen during fitting or tuning.
- **Out-of-fold CV** uses `TimeSeriesSplit` with a **72 h gap** between each train fold and its validation fold. The gap is essential: with a 72 h forecast horizon, adjacent rows share target information, so without it the OOF score would leak. This OOF RMSE — not the test RMSE — is the primary *selection* metric (Section 10).
- **Hyper-parameter tuning** runs inside the same gapped `TimeSeriesSplit` — the classical/tree models via `RandomizedSearchCV` (n_{iter}=8), the gradient boosters via Optuna — fit on the *residual* target (below), so tuning never touches validation or test data.

Residual targets. Models do not predict the absolute future AQI; they predict the *residual* from the current AQI ($y - \text{AQI}_t$), and the pipeline adds AQI_t back. This bakes the strong persistence signal into the target and lets the models focus on the harder change component.

Which features are used. There is no separate feature-selection step: the model trains on *all* columns of the enabled feature groups (Table 3), assembled at the start of each run in `load_and_split`. The set is therefore group-level and can change between runs — `forecast_leads` is **always disabled** to prevent target leakage, and the nightly ablation (Section 8) may switch further groups off.

5.2 Model suite

A deliberately diverse catalogue is trained so the best architecture for this data can be discovered empirically rather than assumed:

- **Classical linear:** Ridge, Lasso, ElasticNet.
- **Tree ensembles:** Random Forest, Gradient Boosting.
- **Gradient boosting (Optuna-tuned):** XGBoost, LightGBM, CatBoost.
- **Hybrid ensembles:** a weighted Voting regressor and a Stacking regressor.
- **Deep learning:** an LSTM, and a distilled MLP that compresses the ensemble.

Every model produces **multi-output (MIMO)** predictions — all three horizons at once (most via `MultiOutputRegressor`; RandomForest natively) — which avoids the error accumulation of recursive forecasting. **CatBoost and LightGBM use the Huber loss**, robust to the heavy-tailed AQI spikes that would otherwise dominate the MSE; XGBoost uses squared error and the LSTM uses MSE.

5.3 Evaluation metrics

Each model is scored with the required regression metrics — **RMSE, MAE and R^2** — plus two domain metrics:

- **Index of Agreement (IoA)** — measures how well the model tracks AQI *movements*; guards against models that merely predict the mean.
- **Skill score** — improvement over the persistence baseline (Section 10).

Metrics are computed both overall and per horizon, and an **out-of-fold (OOF)** RMSE provides a leakage-proof estimate for model selection.

5.4 Champion–Challenger registry

Every run trains many models (the *challengers*); exactly one is promoted to *champion* and deployed. Picking it is multi-criteria via **TOPSIS** — a standard way to rank options that are each good at different things — over (OOF RMSE, IoA, skill score, MAE). Before ranking, three hard gates throw out broken models (`src/training_pipeline/register_model.py`):

```
_MIN_IOA          = 0.15    # minimal statistical agreement (Willmott 1981)
_MIN_SKILL_SCORE  = 0.0     # must beat persistence
_TOPSIS_THRESHOLD = 1.01    # promote only on a >=1% TOPSIS improvement
# Hard gates applied before TOPSIS:
#   1. skill_score > 0       2. NOT overfit_flag       3. IoA >= 0.15
```

The highest-TOPSIS challenger is promoted only if it beats the current champion by at least 1%, preventing needless churn. The model registry lives in MongoDB: `model_metadata` holds the current leaderboard, `training_history` is an append-only log of every model from every run, and only the *champion's* serialised artifacts (model + scaler + feature columns + conformal calibrator + SHAP values, bundled as one ZIP) are stored in GridFS to stay within the storage budget. Keras models (LSTM, distilled MLP) remain leaderboard-only challengers — they are never promoted because the dashboard runtime has no TensorFlow.

Why this selection scheme? Retraining daily on shifting data means a fresh model could be worse than the one in production, so promotion must be guarded: the **hard gates** reject models that are degenerate (worse than persistence, overfit, or barely tracking the signal) before they can ever win, **TOPSIS** then balances metrics that trade off against each other (a model can have the best RMSE but poor agreement), and the **1 % threshold** stops the champion from

flip-flopping on noise. Net effect: the deployed model only ever changes when a genuinely, measurably better one appears.

6 Automated CI/CD Pipeline

Two GitHub Actions workflows (`.github/workflows/`) make the system fully autonomous — no manual runs, no servers.

Table 4: Scheduled workflows.

Workflow	Schedule (cron)	Steps
`feature_pipeline.yml`	`0 * * * *` (hourly)	Fetch data → engineer features → precompute 3-day forecast for the dashboard.
`training_pipeline.yml`	`0 2 * * *` (daily, 02:00 UTC)	Ablation backfill → fill targets → train suite → register champion → upload MLflow artifacts.

Why two cadences? The split is a project requirement: the brief specifies **hourly data ingestion** and a **daily training pipeline**. It also maps cleanly onto the work each job does — the hourly job keeps the dashboard current with fresh inputs, while the daily job keeps the model current by retraining the full suite.

6.1 What the workflows do

Each workflow is a short, declarative list of steps that GitHub runs on a clean Ubuntu VM; there is no server to maintain. The steps map one-to-one onto the project's scripts:

`feature_pipeline.yml` **(hourly,** `0 * * * *`**).**

1. `actions/checkout` — clone the repo onto the runner.
2. `actions/setup-python` — install Python 3.11 with a pip cache.
3. `pip install -r requirements-feature.txt` — the lightweight dependency set (no TensorFlow).
4. Run `src/feature_pipeline/pipeline.py` — fetch the latest observation, engineer the features, and upsert the row into `aqi_features`.
5. Run `src/feature_pipeline/predict.py` — recompute the 3-day forecast and write it to the `predictions` collection the dashboard reads.

`training_pipeline.yml` **(daily,** `0 2 * * *`**).**

1. Checkout + Python 3.11, then `pip install -r requirements.txt` (the full set, incl. TensorFlow).
2. Run `src/backfill/ablation_backfill.py` — compare imputation strategies and persist the scores.
3. Run `src/backfill/fill_targets.py` — fill in targets for rows whose future is now observed.
4. Run `src/training_pipeline/train_model.py` — train the model suite, evaluate, and register the champion.
5. `actions/upload-artifact` — save the run's `mlruns/` MLflow logs (30-day retention).

Every step that touches the database or an API receives the four secrets through its `env:` block (detailed next). The result is fully unattended: at the time of writing the repository's **Actions** tab shows 471 successful scheduled runs — each hourly feature run finishing in about a minute and each daily training run in about twelve (Figure 2). A reviewer can open the tab and verify all of this with no credentials.

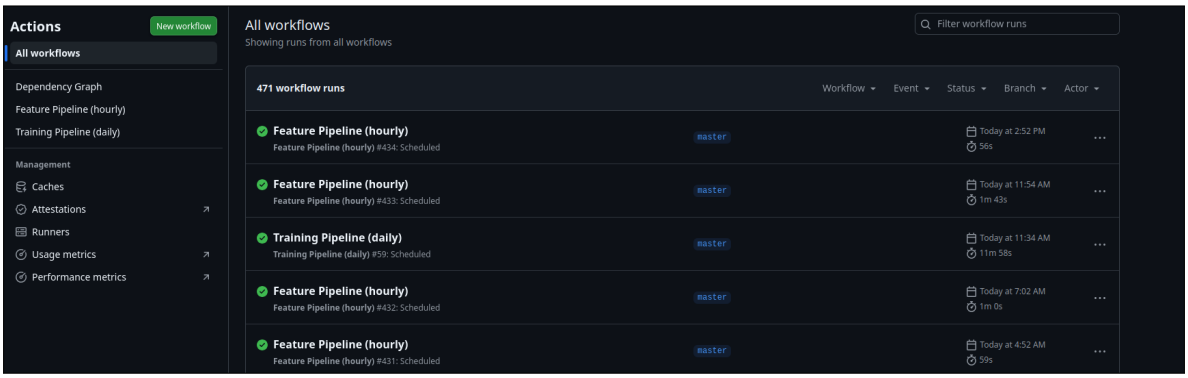


Figure 2: The repository's GitHub Actions tab: the hourly *Feature Pipeline* and daily *Training Pipeline* run on schedule and complete green, with per-run durations. The full history is public, so the automation is verifiable without any credentials.

6.2 How GitHub Actions connects to MongoDB and the APIs

Connecting a cloud-hosted, credential-free runner to a private Atlas cluster takes two deliberate steps:

1. **Credentials as repository secrets.** The four secrets (`AQICN_API_KEY`, `OPENWEATHER_API_KEY`, `MONGODB_URI`, `MONGODB_DB_NAME`) are stored under *Settings* → *Secrets and variables* → *Actions* and injected into each step's `env:` block (e.g. `MONGODB_URI: ${{ secrets.MONGODB_URI }}`). They never appear in the repository, the logs, or the code — `db.py` simply reads `os.environ["MONGODB_URI"]` at run time.
2. **Atlas network access for dynamic runner IPs.** GitHub-hosted runners get a fresh, unpredictable public IP on every run, so the Atlas cluster's *Network Access* list must allow `0.0.0.0/0` (allow-from-anywhere). Without this the SRV connection times out; it is the single configuration that actually lets the Actions runner reach the database. (Authentication still gates access via the SRV user/password embedded in `MONGODB_URI`.)

Runner specifics. Both jobs run on `ubuntu-latest` with Python 3.11 and a pip cache. The hourly feature job installs the lightweight `requirements-feature.txt` (no TensorFlow) and is capped at 15 min; the daily training job installs the full `requirements.txt` and is allowed up to 120 min. The training run uploads its `mlruns/` directory as a build artifact (30-day retention). Both expose a `workflow_dispatch` manual trigger. Because the run history is public on the repository's **Actions** tab, a reviewer can verify the automation without any credentials.

7 Web Application Dashboard

The dashboard (`app.py` + `src/dashboard/`) is a Streamlit application deployed on Streamlit Community Cloud:

aqi-predictor-clgefbv7tu87ehnluaddsu.streamlit.app

It reads precomputed results from MongoDB — it never trains or runs heavy inference live — and organises them into four tabs:

1. **Live Forecast** — current + 24/48/72 h AQI gauges (EPA colour zones), the 3-day forecast with conformal uncertainty bands, and a hazardous-AQI alert.
2. **Leaderboard** — every model from the latest run ranked by RMSE and TOPSIS, with the champion highlighted; this *is* the model registry view.
3. **Ablation Study** — auto-updated charts showing which feature groups help or hurt accuracy.
4. **Feature Importance** — a SHAP bar chart for the best tree-based model.

Forecasts are precomputed hourly by the feature pipeline and cached with Streamlit's `@st.cache_data` (≈10 min TTL), so in-session use is fast; only a cold start (idle app spin-up) takes 10–20 s.

7.1 Navigating the dashboard

This section walks through the live app as a reviewer would use it, with screenshots captured from the deployed dashboard.

Sidebar (always visible). The left sidebar shows the active city, a **Refresh data** button that clears the Streamlit cache and re-reads MongoDB on demand, and a colour legend for the six US-EPA AQI zones used throughout the charts.

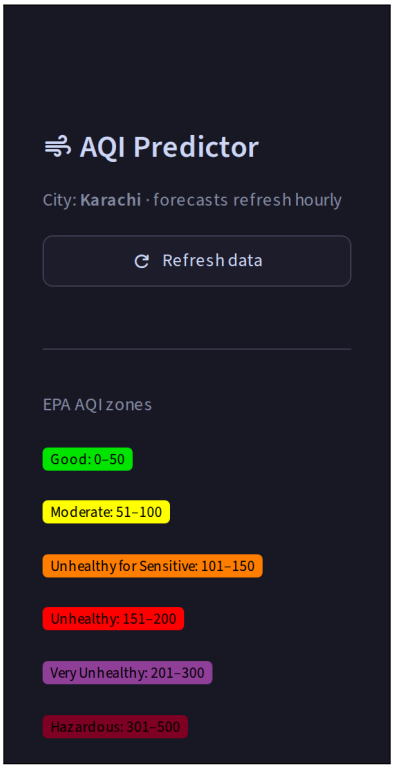


Figure 3: Sidebar: city selector, manual *Refresh data* control, and the EPA AQI colour-zone legend.

Tab 1 — Live Forecast. The default tab. Four gauges across the top show the current AQI and the 24/48/72 h forecasts, each coloured by EPA zone. A red banner appears when any horizon crosses the hazardous threshold (AQI ≥ 150). Below, an expandable "How to read these charts" note precedes four time-series panels: the observed AQI over the last 7 days; the *forecast track record* (past forecasts scored against what was actually observed — an honest, leakage-free accuracy view); the champion hindcast across history; and the live next-72 h forecast with a 90 % conformal band on the 24 h horizon. A caption reports which model produced the forecast and when.

Figure 4: Live Forecast tab (top): the four EPA-coloured AQI gauges — current AQI and the 24/48/72 h forecasts — above the observed AQI over the last 7 days.

Figure 5: Live Forecast tab (middle): the *forecast track record* (past forecasts scored against the AQI later observed — an honest, leakage-free accuracy view) above the *champion hindcast* (the champion's 24 h-ahead out-of-fold predictions across history).

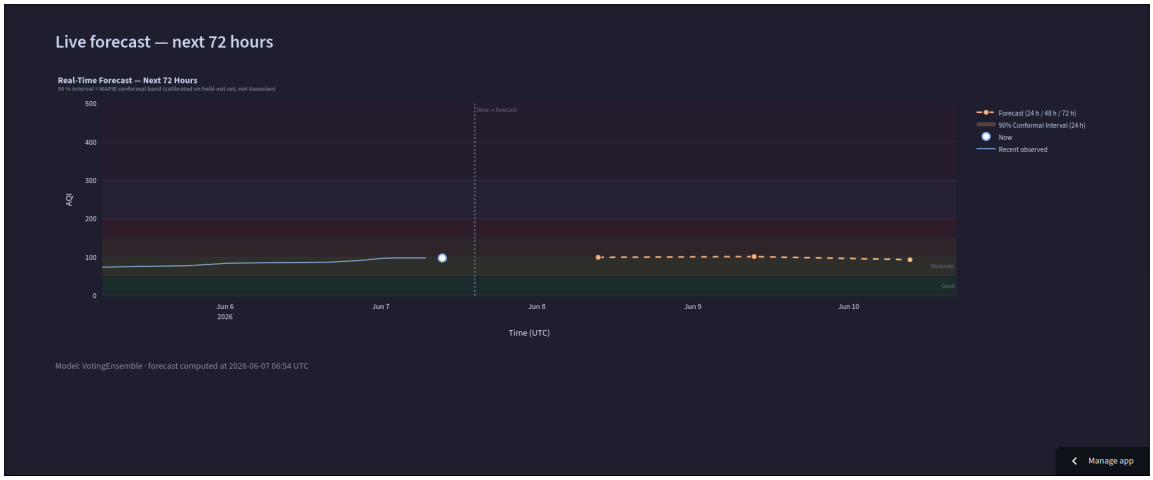


Figure 6: Live Forecast tab (bottom): the next-72 h live forecast. The blue line is recent observed AQI, the dashed points are the 24/48/72 h forecasts, and the shaded band is the 90 % MAPIE conformal interval on the 24 h horizon; the caption records the model and the time the forecast was computed.

Tab 2 — Leaderboard. This *is* the model registry, rendered live from `model_metadata`. A highlighted champion card shows its TOPSIS, RMSE, OOF RMSE, IoA and skill score; an expandable glossary defines each metric and its standard; a radar chart compares the top models; and a full table lists every model from the latest run (champion row in gold, negative skill scores flagged in red).

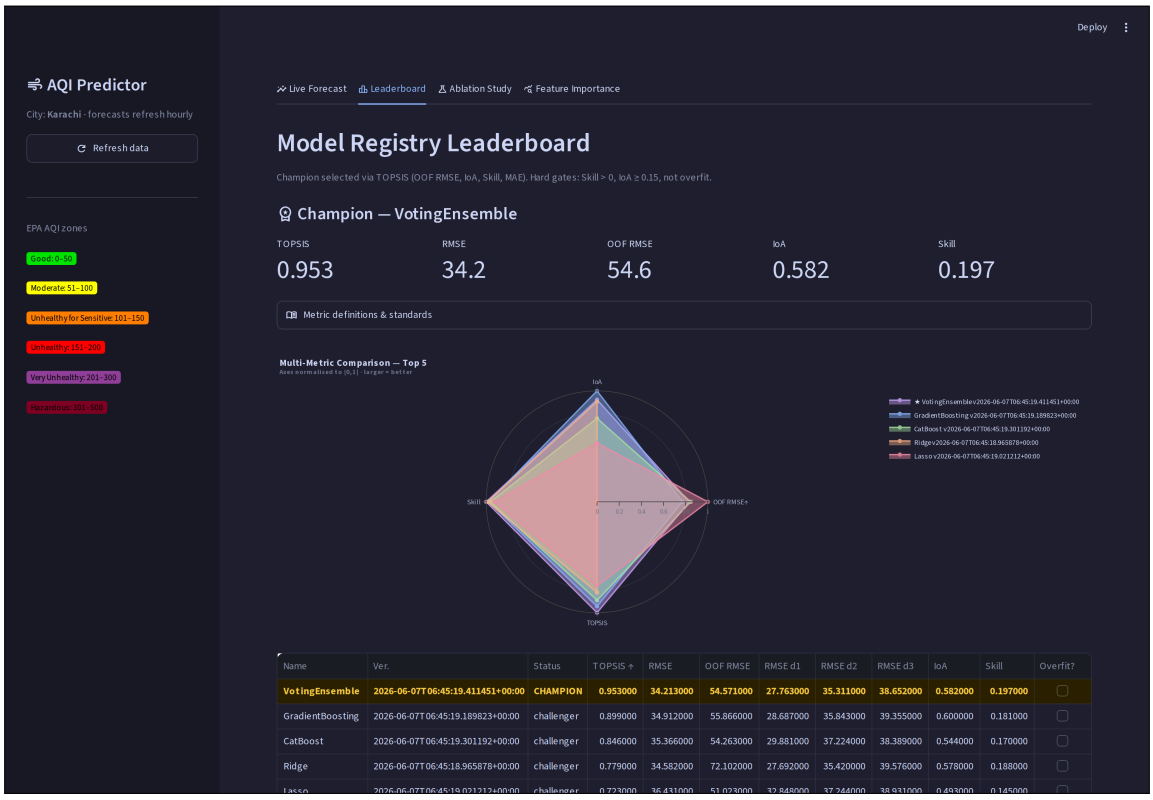


Figure 7: Leaderboard tab: champion summary card, metric radar, and the full TOPSIS-ranked model table.

Tab 3 — Ablation Study. Two auto-updating charts (refreshed nightly after training): the RMSE *increase* when each feature group is removed (taller bars = more important groups), and

the backfill imputation-strategy comparison (Section 4.1).

Figure 8: Ablation Study tab: per-feature-group importance and the imputation-strategy RMSE comparison.

Tab 4 — Feature Importance. A SHAP bar chart of the top-15 features for the best tree-based model, quantifying each feature's mean |SHAP| contribution to the prediction.

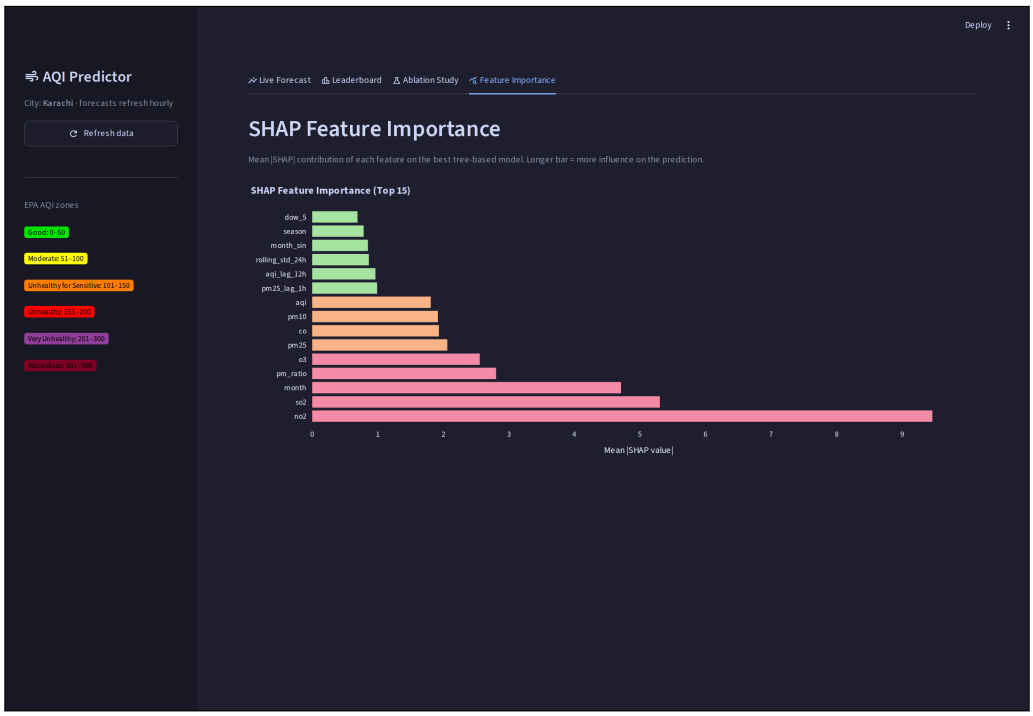


Figure 9: Feature Importance tab: SHAP attribution for the best tree-based model's top features.

8 Advanced Analytics

Beyond core forecasting, the system implements the full advanced-analytics brief.

8.1 Exploratory Data Analysis

`notebooks/eda.ipynb` loads the feature-store rows (via `fetch_training_data()`) and characterises them. The figures below are exported from that notebook into `report/figures/`.

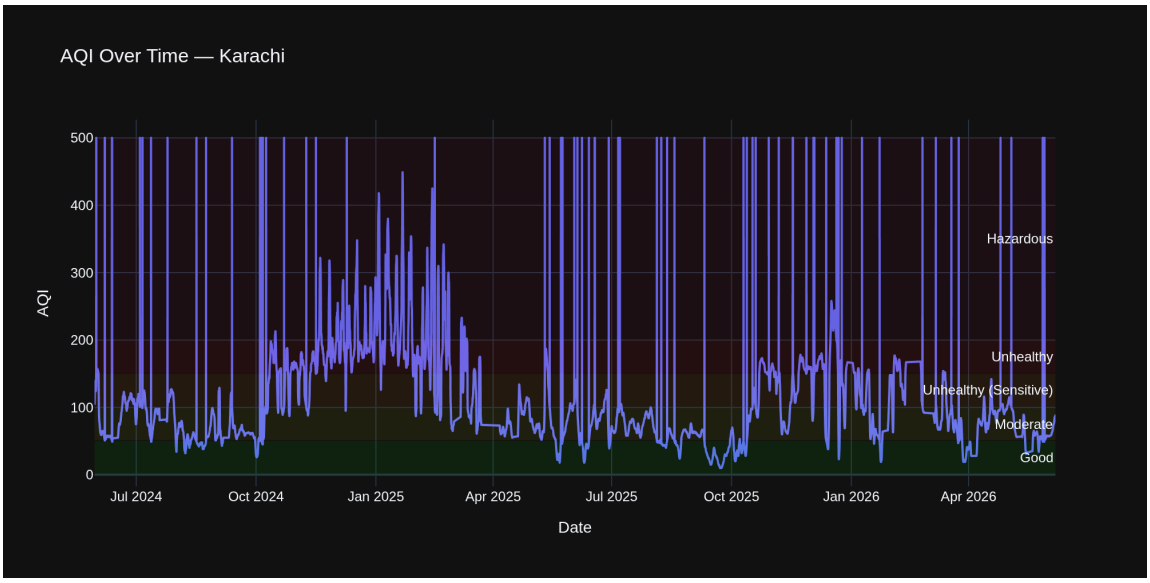


Figure 10: AQI over the full history, with EPA health-zone bands. Karachi spends long stretches in the *unhealthy* and *hazardous* ranges, with a strong seasonal envelope.

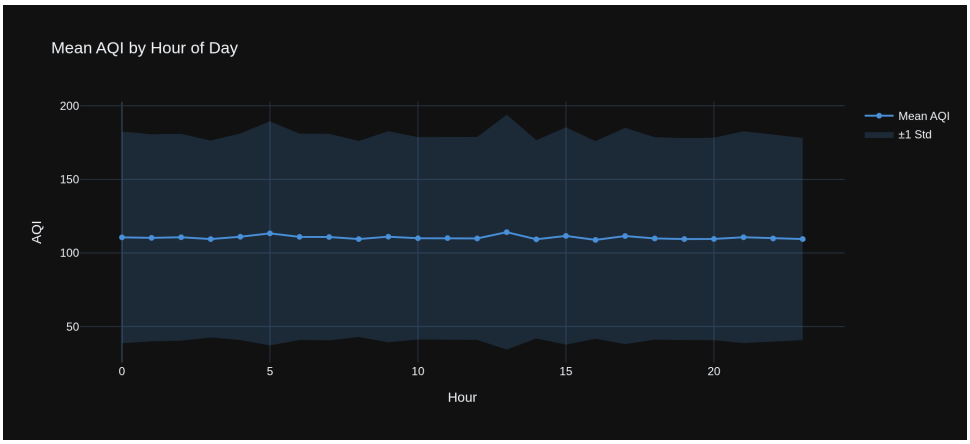


Figure 11: Mean AQI by hour of day (±1 SD), showing a clear diurnal cycle that motivates the cyclic (Fourier) time features.

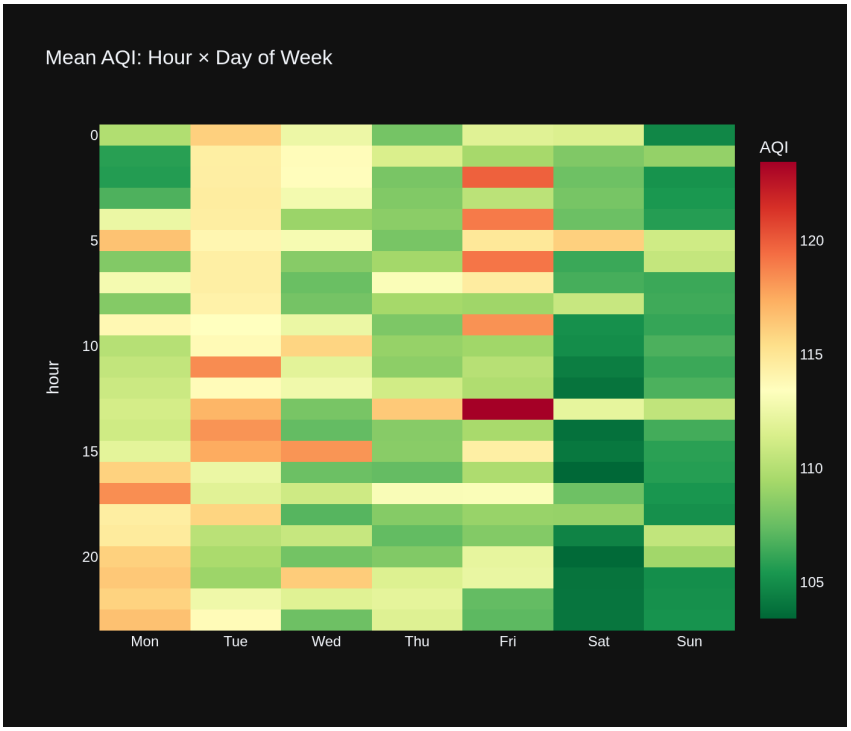


Figure 12: Mean AQI across hour × day-of-week.

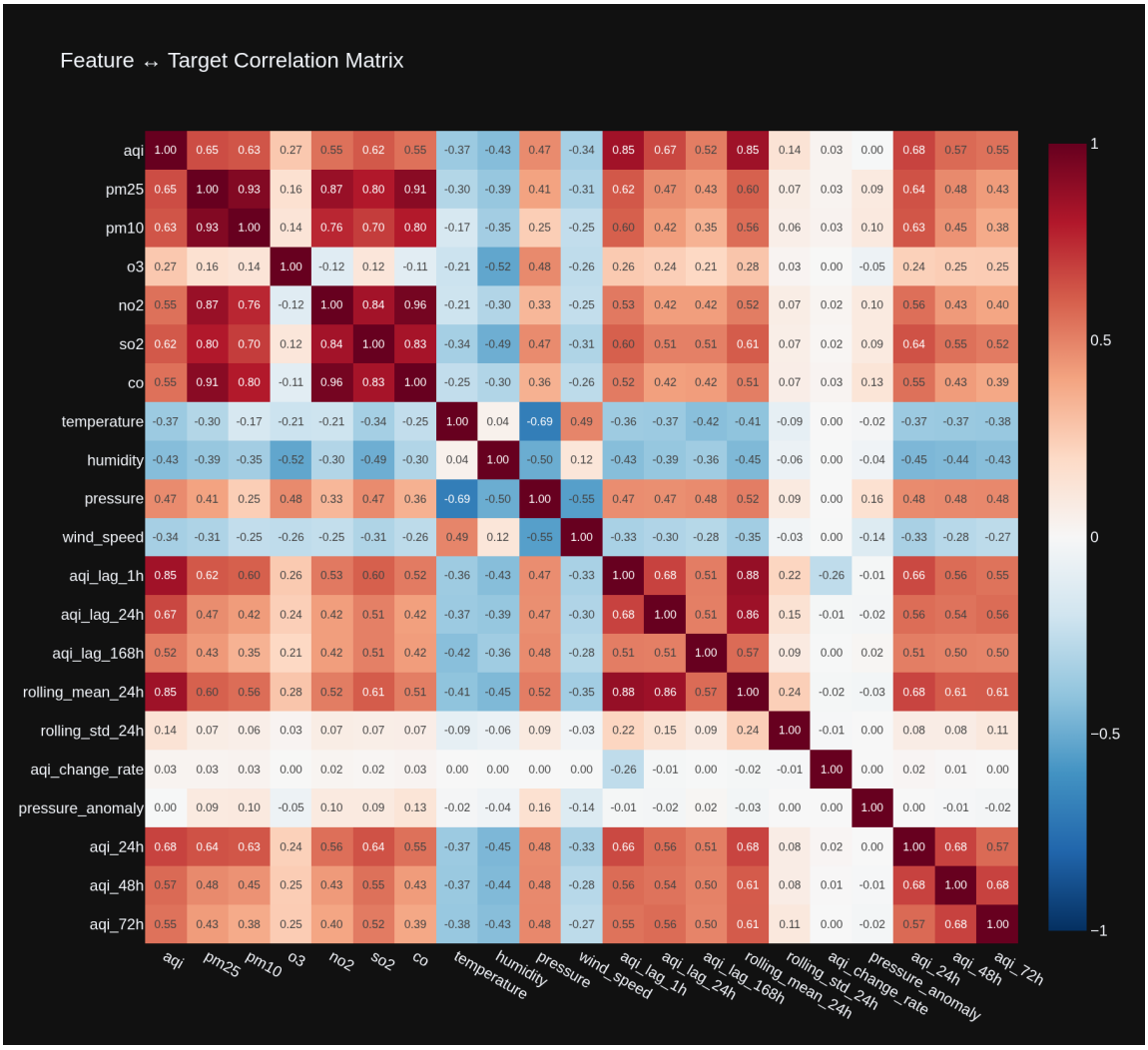


Figure 13: Correlation between representative features (one per group) and the three forecast targets. Recent AQI lags and the 24 h rolling mean correlate most strongly with the targets; meteorology adds weaker, complementary signal.

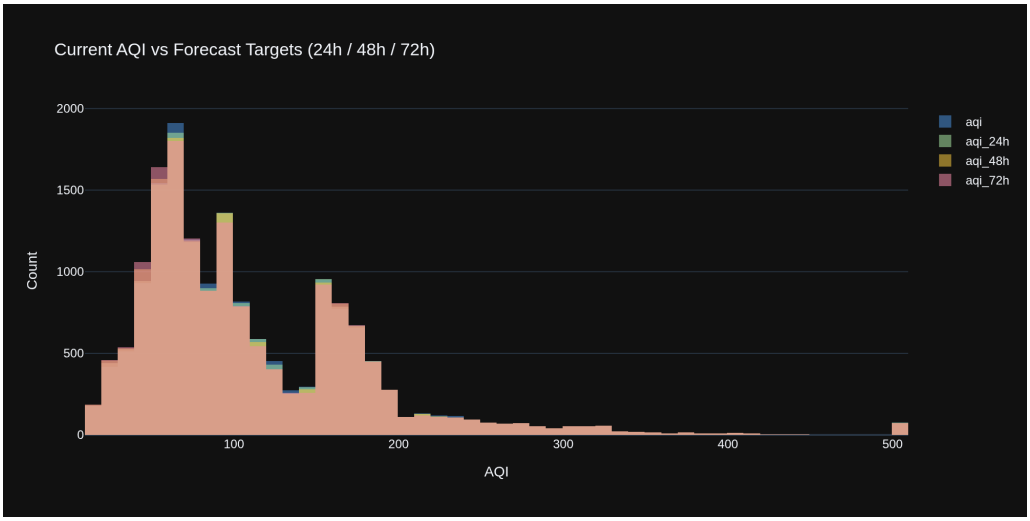


Figure 14: Distribution of current AQI versus the 24/48/72 h targets. They share the same right-skewed shape — justifying a single MIMO model across horizons.

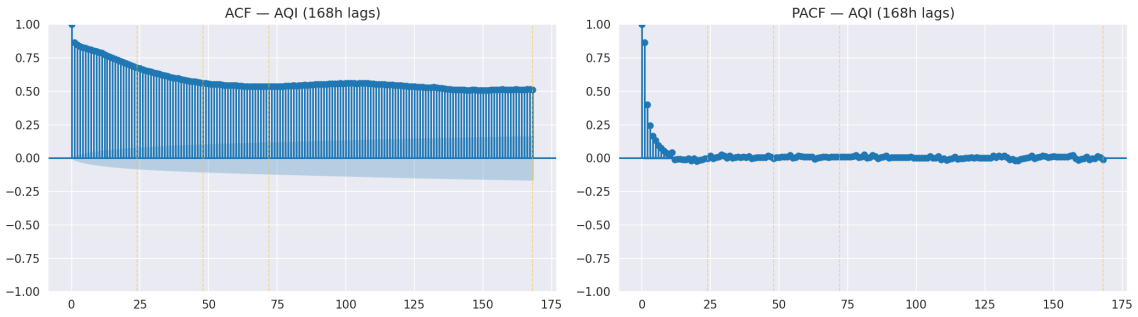


Figure 15: ACF/PACF of AQI out to 168 h. Pronounced spikes at the 24 h (daily) and 168 h (weekly) lags justify the multi-scale lag features.

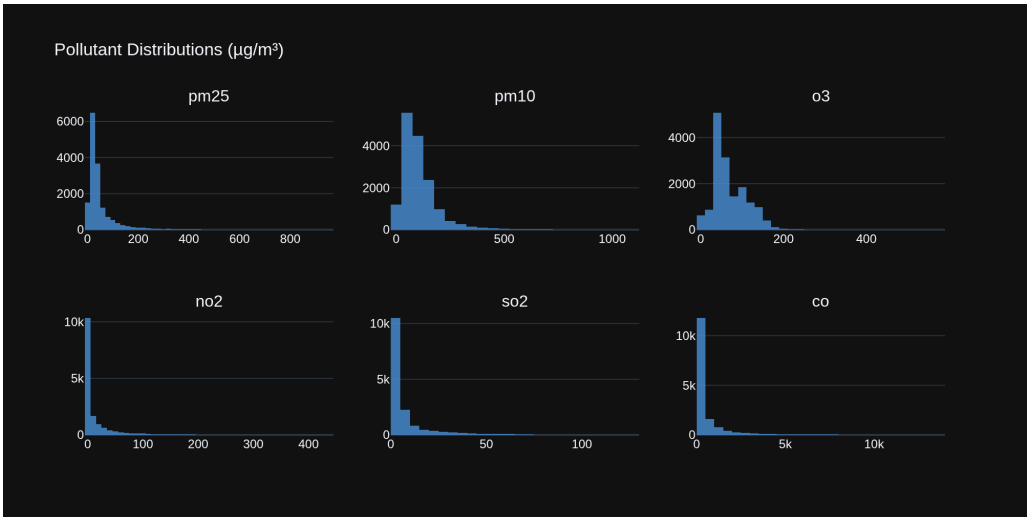


Figure 16: Pollutant concentration distributions. $PM_{2.5}$ and PM_{10} are strongly right-skewed, motivating the $\log(x+1)$ transform.

8.2 Explainability — SHAP

The dashboard's Feature Importance tab renders SHAP values for the champion (Figure 9), attributing each forecast to its drivers. The strongest contributions come from the **pollutant concentrations** (NO_2, SO_2, O_3, $PM_{2.5}$/PM_{10}, CO) and **seasonality** (`month`, `season`), with the current AQI, recent AQI/$PM_{2.5}$ lags and the 24 h rolling spread (`rolling_std_24h`) contributing further — a sensible mix of "what the air is made of right now" and "what time of year it is".

8.3 Hazardous-AQI alerts

When any horizon's forecast exceeds the configured threshold (`AQI_ALERT_THRESHOLD = 150`, the EPA *unhealthy* boundary), the dashboard raises a prominent banner so a user sees the warning immediately.

8.4 Conformal prediction intervals

Point forecasts alone hide uncertainty. Using MAPIE (`src/training_pipeline/conformal.py`), the champion is wrapped with a conformal calibrator that produces statistically valid prediction intervals, shown as shaded bands around the forecast line.

8.5 From statistical to deep learning

The model suite spans the full requested spectrum: statistical/linear (Ridge, Lasso, ElasticNet), tree ensembles, gradient boosting, hybrid ensembles, and deep learning (LSTM + distilled MLP) — all compared on the same leaderboard.

9 Feature Store & Model Registry (MongoDB)

In place of Hopsworks or Vertex AI, this project uses **MongoDB Atlas** as both the feature store and the model registry. A single managed, free-tier database keeps the architecture simple and credential-free for reviewers (the dashboard surfaces both live). Table 5 lists the collections.

9.1 Connection model

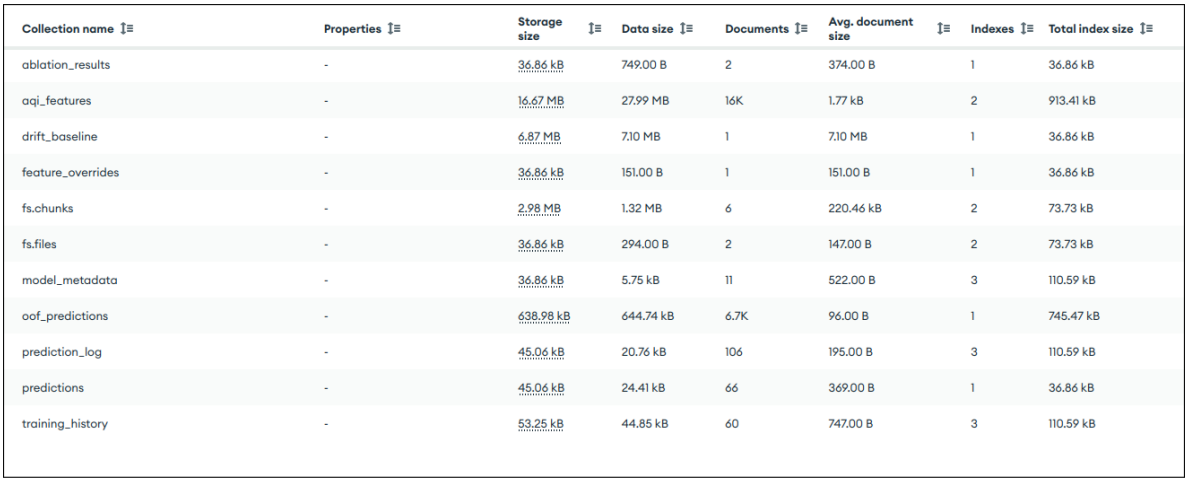
All database access goes through one helper (`src/db.py`). A **thread-safe `MongoClient` singleton** is created lazily from the `MONGODB_URI` environment variable (an Atlas SRV string) with `serverSelectionTimeoutMS=10000`; the client manages its own connection pool, so it is shared across the feature, prediction and training code without re-connecting. The database name comes from `MONGODB_DB_NAME` (default `aqi_db`), and champion artifacts are stored through `gridfs.GridFS` on the same database. No connection string is ever committed — it is supplied via environment/secret only (Sections 6, 11).

Table 5: MongoDB collections.

Collection	Purpose	Write pattern
`aqi_features`	Feature store: one engineered row per city/hour + targets	upsert by (city,ts)
`model_metadata`	Live leaderboard: current metrics per model	upsert by model_name
`training_history`	Append-only log of every model, every run	insert_many
`oof_predictions`	Out-of-fold hindcast for the champion	upsert
`predictions`	Precomputed 3-day forecast (hourly)	upsert
`prediction_log`	Rolling log of issued forecasts (TTL-expired)	insert
`ablation_results`	Feature-group ablation scores	upsert
`feature_overrides`	Ablation-driven feature-group toggles	upsert
`fs.files` / `fs.chunks`	GridFS: champion model + scaler + calibrator	store/replace

9.2 Indexes

Figure 17 is the live Atlas collections view — every collection with its document and index counts. Table 6 is the full index inventory; every collection additionally carries MongoDB's default unique `_id_` index. The per-collection index screenshots from Atlas follow.



Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
ablation_results	-	36.86 kB	749.00 B	2	374.00 B	1	36.86 kB
aqi_features	-	16.67 MB	27.99 MB	16K	1.77 kB	2	913.41 kB
drift_baseline	-	6.87 MB	7.10 MB	1	7.10 MB	1	36.86 kB
feature_overrides	-	36.86 kB	151.00 B	1	151.00 B	1	36.86 kB
fs.chunks	-	2.98 MB	1.32 MB	6	220.46 kB	2	73.73 kB
fs.files	-	36.86 kB	294.00 B	2	147.00 B	2	73.73 kB
model_metadata	-	36.86 kB	5.75 kB	11	522.00 B	3	110.59 kB
oof_predictions	-	638.98 kB	644.74 kB	6.7K	96.00 B	1	745.47 kB
prediction_log	-	45.06 kB	20.76 kB	106	195.00 B	3	110.59 kB
predictions	-	45.06 kB	24.41 kB	66	369.00 B	1	36.86 kB
training_history	-	53.25 kB	44.85 kB	60	747.00 B	3	110.59 kB

Figure 17: Atlas collections overview: storage, document counts and index counts across all collections.

Table 6: Index inventory (secondary indexes, beyond the default `_id_`).

Collection	Secondary index(es)	Properties
`aqi_features`	`city_1_timestamp_1`	unique, compound
`model_metadata`	`model_name_1`, `is_champion_1`	single-field
`training_history`	`model_name_1_trained_at_-1`, `run_id_1`	compound; single
`prediction_log`	`logged_at_1`, `city_1_ts_epoch_-1`	TTL; compound
`fs.files`	`filename_1_uploadDate_1`	compound (GridFS)
`fs.chunks`	`files_id_1_n_1`	unique, compound (GridFS)

Others (`predictions`, `oof_predictions`, `ablation_results`, `feature_overrides`): default `_id_` index only.

What each key index is for.

- `aqi_features` — the **unique compound** `(city, timestamp)` index is the upsert key for the whole pipeline (`UpdateOne({city, timestamp}, ..., upsert=True)`), which is what makes the backfill and hourly runs idempotent — re-running never duplicates an hour. **It is created once in the Atlas UI** (Setup Step 3); the application requires it but does not create it in code.
- `training_history` — created in code by `_ensure_history_indexes()` on every run: compound `(model_name, trained_at DESC)` for per-model history and `run_id` for grouping a run's models. Kept indefinitely (no TTL).
- `model_metadata` — `model_name_1` is the upsert/lookup key (the most-used index) and `is_champion_1` gives a direct champion lookup for the dashboard.
- `prediction_log` — a **TTL** index on `logged_at` auto-expires old forecast log entries, and the compound `(city, ts_epoch DESC)` serves recent-first reads.
- `fs.files` / `fs.chunks` — the standard GridFS indexes created by the driver.

Figure 18: `aqi_features` indexes — the unique compound `(city, timestamp)` key.

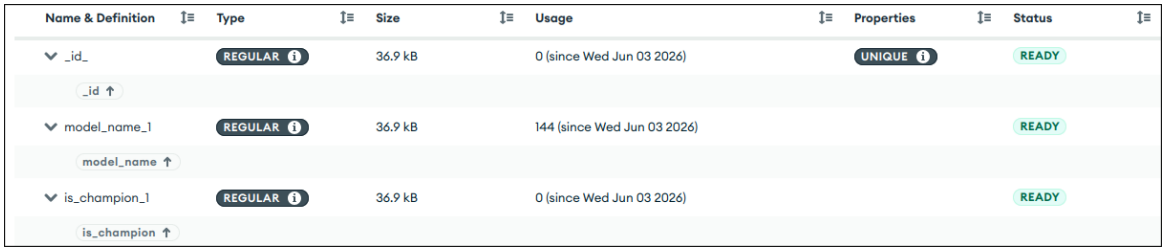


Figure 19: `model_metadata` indexes — `model_name` and `is_champion`.

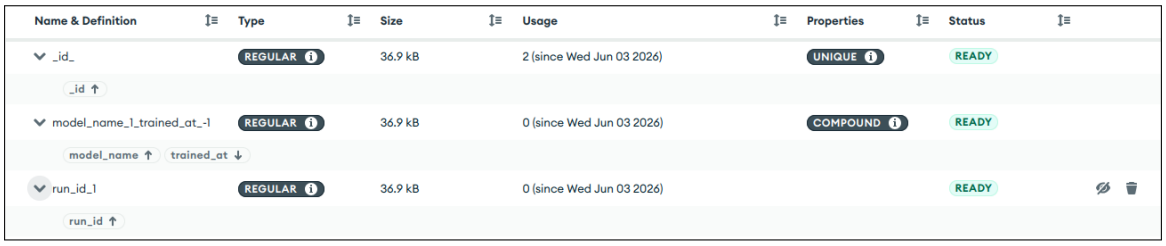


Figure 20: `training_history` indexes — compound `(model_name, trained_at)` and `run_id`.

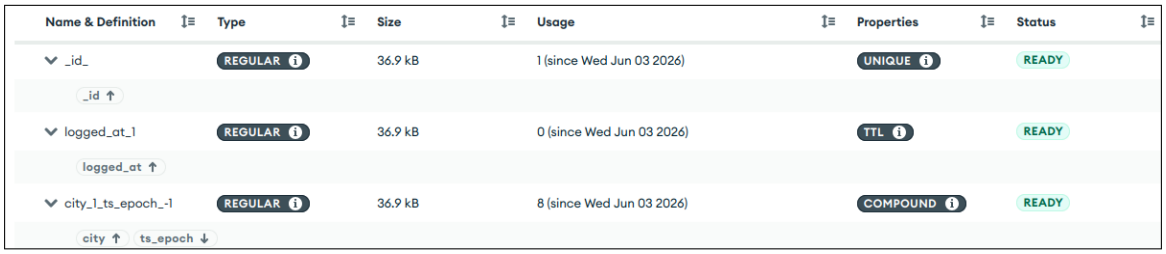


Figure 21: `prediction_log` indexes — a TTL index on `logged_at` plus compound `(city, ts_epoch)`.

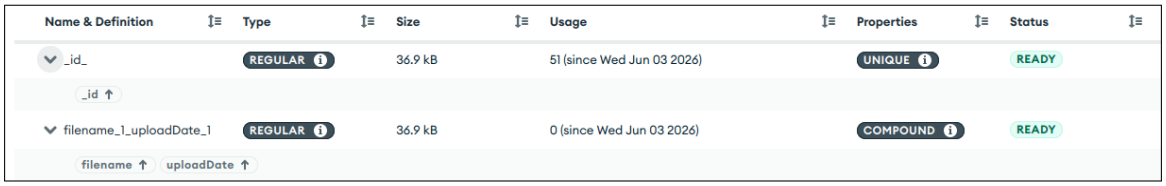


Figure 22: GridFS `fs.files` indexes.

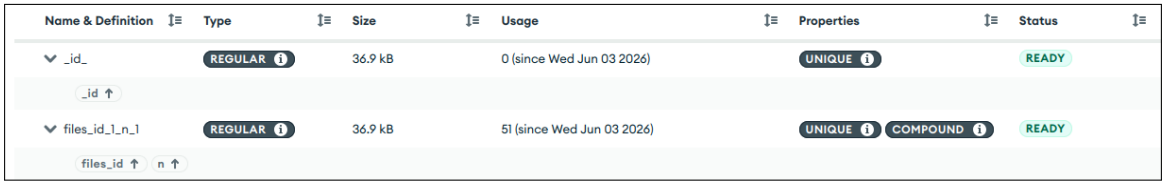


Figure 23: GridFS `fs.chunks` indexes.

```
# training_history - created automatically each run (register_model.py)
col.create_index([("model_name", 1), ("trained_at", -1)])
col.create_index("run_id")

# aqi_features - created once in the Atlas UI (Setup Step 3):
#   db.aqi_features.createIndex({city: 1, timestamp: 1}, {unique: true})
```

9.3 Document schemas

`aqi_features` (feature store). One document per (city, hour): the identity fields, the current `aqi` ($PM_{2.5}$-derived, EPA), the raw pollutant and meteorology fields, the ≈50 engineered columns grouped exactly as in Table 3, and the three regression targets. Compact shape:

```
{ city: "karachi", timestamp: ISODate,
  aqi: 100,                                  # current EPA AQI (24h-mean PM2.5)
  pm25,pm10,o3,no2,so2,co,                   # raw_pollutants
  temperature,humidity,pressure,wind_speed,visibility,cloud_cover,
      precipitation_1h, # meteorology
  hour,month,season,is_weekend,dow_0..dow_6, # time_basic
  hour_sin,hour_cos,month_sin,month_cos,     # time_fourier
  aqi_lag_{1,3,6,12,24,48,72,168}h, pm25_lag_{1,3,6,12,24}h,   # lag_features
  rolling_mean_{3,6,24}h, rolling_std_24h, rolling_min_24h, rolling_max_24h,
      aqi_pct_change_24h,
  mixing_height_proxy, wind_transport_effect, temp_humidity_interaction,
      pm_ratio,
  wind_dir_sin, wind_dir_cos, pressure_anomaly, aqi_change_rate,   # physics/
      cross
  aqi_24h, aqi_48h, aqi_72h }                # targets (24h-mean future AQI)
```

`predictions` / `model_metadata`. `predictions` holds the precomputed 3-day forecast per city: the current AQI, the 24/48/72 h point forecasts with their conformal lower/upper bounds, the model version, and the time it was computed. `model_metadata` is the live leaderboard — one document per model with its flat metrics (`rmse`, `oof_rmse`, `mae`, `r2`, `ioa`, `skill_score`, `topsis_score`), the `is_champion` flag, and (for the champion) its `feature_cols` and `artifacts_gridfs_id`.

9.4 The `training_history` document

Each daily run inserts one document *per model* (including the leaderboard-only LSTM and distilled MLP) via `insert_many` — documents are never overwritten, so the collection is a complete audit log. Each document carries:

- **Run identity:** `run_id` (`GITHUB_RUN_ID` or a UUID), `trained_at` (one shared ISO timestamp for the run), `github_sha`, `github_run_number`.
- **Run context:** `feature_groups` (snapshot of which groups were enabled), `feature_count`.
- **Per model:** `model_name`, `model_type`, `champion_eligible`, `promoted` (true only for the model promoted that run), `topsis_score`.
- **Metrics** (flattened): `rmse`, `oof_rmse`, `mae`, `r2`, `ioa`, `skill_score`, per-horizon `rmse_d1/d2/d3`, `overfit_flag`.

How to access/verify (no credentials needed): the model registry is rendered live in the dashboard *Leaderboard* tab from `model_metadata`; the feature store drives both the live forecast and the EDA notebook. The full per-run history is preserved in `training_history` (60 documents across the training runs to date).

10 Results & Evaluation

Two terms are used throughout this section:

- **Persistence baseline** — the simplest possible forecast: assume the air will not change, i.e. predict that the AQI in h hours equals the AQI right now. It needs no model at all, so it is

the bar any real model must beat. A model's *skill score* is exactly its percentage improvement in error over this baseline.

- **Champion model** — the single best model, chosen automatically each day from all the models trained that run (the "challengers") and deployed to produce the live forecasts. "Champion–challenger" just means: keep training many candidates, but only ever promote the one that wins (Section 5).

Baseline. Table 7 gives the persistence baseline's error on the full history. As expected, it gets worse the further ahead we look — guessing 3 days out from today's value is much weaker than guessing 1 day out.

Table 7: Persistence baseline on the full history (15,673 paired rows).

Horizon	RMSE	MAE
24 h	56.28	26.75
48 h	65.35	36.08
72 h	67.22	39.39

Champion. The current champion — a **Voting ensemble** (the average of several models, promoted 2026-06-04) — beats persistence at every horizon (Table 8, Figure 24): it roughly halves the day-ahead error. It is trained on all columns of the enabled feature groups (Table 3; the `forecast_leads` group is always excluded to avoid leakage) and predicts the *residual* from the current AQI (how much the air will change), which the pipeline adds back to today's value. Accuracy decays with the horizon (R^2 from 0.38 at 24 h to 0.03 at 72 h), reflecting the genuine difficulty of three-day-ahead forecasting.

Table 8: Champion per-horizon performance (temporal test set).

Horizon	RMSE	MAE	R^2
24 h (Day 1)	28.26	13.61	0.385
48 h (Day 2)	35.27	20.56	0.152
72 h (Day 3)	37.99	23.93	0.033
Overall	**34.07**	**19.33**	**0.182**

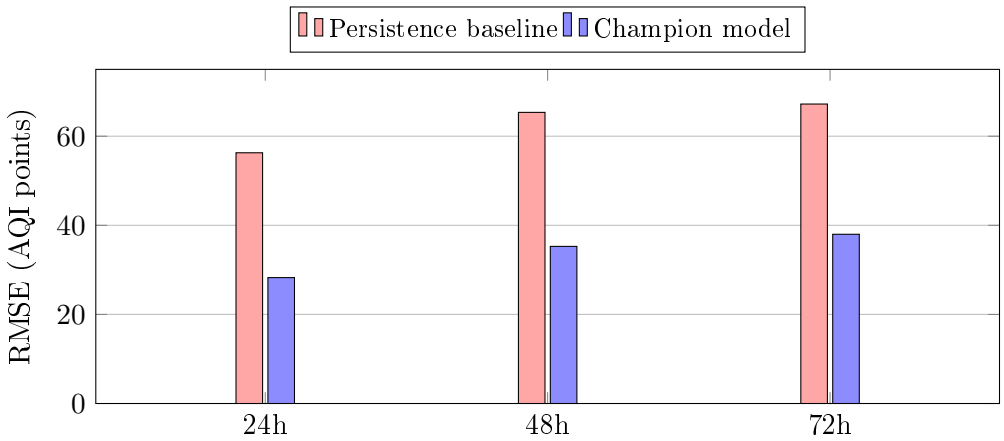


Figure 24: Champion RMSE versus the persistence baseline at each horizon. The champion roughly halves the error at 24 h and retains a clear margin at 72 h.

Leaderboard. Table 9 is a representative run's leaderboard, TOPSIS-ranked. The Voting ensemble, CatBoost and Gradient Boosting cluster at the top; the distilled MLP fails the skill-score gate and is excluded from promotion.

Table 9: Representative training-run leaderboard (ranked by TOPSIS). Champion in bold.

Model	RMSE	OOF	MAE	R^2	IoA	TOPSIS
VotingEnsemble (champion)	**34.07**	56.92	19.33	0.182	0.596	**0.925**
CatBoost	34.44	56.18	19.56	0.165	0.577	0.915
GradientBoosting	34.91	55.87	20.17	0.142	0.600	0.879
XGBoost	35.39	58.58	20.36	0.118	0.595	0.813
Ridge	34.58	72.10	20.39	0.158	0.578	0.753
Lasso	36.43	51.02	19.76	0.065	0.493	0.699
RandomForest	37.73	56.14	22.34	−0.003	0.553	0.551
ElasticNet	40.40	52.22	20.22	−0.149	0.536	0.315
LightGBM	42.06	54.28	20.28	−0.246	0.537	0.215
DistilledMLP	63.93	63.93	52.15	−1.879	0.387	gated

The OOF RMSE (≈56) is markedly higher than the test RMSE (≈34) because it is the conservative, leakage-proof estimate used for *selection*; the test-set figures in Table 8 reflect deployed performance. Either way, the champion improves substantially on persistence (skill score ≈ 0.20).

Note on figures. The numbers in Tables 8–9 are from one representative daily run. Because the system retrains every night, the *live* dashboard (Figures 6 and 7) shows the current run, so its exact values will differ slightly from these tables.

11 Deployment & Reproducibility

The full setup is documented in the repository `README.md`. In brief: register AQICN/OpenWeather keys, create a free MongoDB Atlas cluster, add the four secrets to GitHub, run the backfill once and the training once to seed the database, then deploy `app.py` to Streamlit Community Cloud. After that the two GitHub Actions workflows keep everything current automatically. The pinned `requirements.txt` and `runtime.txt` (Python 3.11) make runs reproducible; MLflow records each training run's parameters and metrics.

12 Conclusion

The project delivers a complete, autonomous AQI-forecasting system covering every required deliverable — a feature pipeline, historical backfill, a multi-model training pipeline with a champion–challenger registry, hourly/daily CI/CD, a public dashboard, and an advanced-analytics layer (EDA, SHAP, alerts, conformal intervals and ablation) — all deployed on free-tier cloud services. The champion roughly halves the day-ahead error relative to persistence and degrades gracefully out to three days.

A Deliverables → Implementation Mapping

Required deliverable	Implementation
Fetch raw weather/pollutant data (OpenWeather/Open-Meteo/AQICN)	`src/feature_pipeline/fetch_data.py`
Time-based + derived features (incl. AQI change rate)	`src/feature_pipeline/feature_engineering.py`
Store features in a feature store	MongoDB `aqi_features` via `store_features.py`
Historical backfill / training dataset	`src/backfill/` (`backfill_features.py`, `fill_targets.py`)
Fetch features/targets for training	`fetch_training_data()` in `store_features.py`
Multiple models (RF, Ridge, TF, ...)	`src/training_pipeline/models.py`
RMSE / MAE / R^2 evaluation	`src/training_pipeline/evaluate_model.py`
Model registry	MongoDB `model_metadata`, `training_history`, GridFS (`register_model.py`)
Hourly feature pipeline (CI/CD)	`.github/workflows/feature_pipeline.yml`
Daily training pipeline (CI/CD)	`.github/workflows/training_pipeline.yml`
Real-time 3-day predictions	`predict.py` + `predictions` collection
Interactive dashboard	`app.py` + `src/dashboard/` (Streamlit)
EDA / trend analysis	`notebooks/eda.ipynb`
SHAP / feature importance	`src/dashboard/components/shap_plot.py`
Hazardous-AQI alerts	`src/dashboard/components/alerts.py`
Multiple forecasting models (statistical → DL)	full model suite (Section 5)